



전자전기컴퓨터설계실험II

실험 전 보고서

FPGA 응용회로 실습

실험 19-25

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 09월 27일
대상	PWM LED 밝기 · RGB LED PWM · 피에조 단일 음 · 스텝모터 위상 제어 · MM:SS 시계 · 문자 LCD 제어 · PC-FPGA UART 에코
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab3>

제출 태그: [lab3-pre-v1](#)

커밋 해시: [26acfe878b6d7e0b0355c3be7cb1d638fb124d1c](#)

```

tag lab3-pre-v1
Tagger: a40093632-ai <a40093632@gmail.com>
TaggerDate: Mon Sep 28 02:29:34 2026 +0900

LAB3 pre-lab submission

commit 26acfe8786d7e0b8355c3be7cb1d638fb124d1c (HEAD -> main, tag: lab3-pre-v1, origin/main, origin/HEAD)
Author: a40093632-ai <a40093632@gmail.com>
AuthorDate: Sun Sep 27 17:57:02 2026 +0900
Commit: a40093632-ai <a40093632@gmail.com>
CommitDate: Sun Sep 27 17:57:02 2026 +0900

```

그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

LAB3의 목적은 FPGA의 50MHz 시스템 클록을 기준으로 PWM, 주파수 분주, 모터 위상 제어, 표시장치 제어 및 비동기 직렬 통신 회로를 구현하는 것이다. 앞서 설계한 조합 및 순차논리회로를 기반으로 한다. 모든 순차회로는 하나의 시스템 클록에서 동작하며, 느린 출력 주기와 주변 장치 제어 시점은 내부 카운터가 생성하는 clock-enable을 이용하여 결정한다.

각 회로의 실제 시간 파라미터를 계산하고, Testbench에서는 기능의 순서와 경계 조건을 유지하면서 파라미터를 축소하여 빠르게 검증한다. Icarus Verilog의 PASS 로그와 VaporView 파형으로 예상 결과를 확인하고, 한 항목을 변경한 뒤 나타나는 FAIL 또는 타이밍 변화를 분석한다.

실험 범위

	회로	핵심 기능
19	PWM LED 밝기	버튼을 누를 때마다 듀티비를 10%씩 변경하고 LED 8개에 동일한 PWM 출력
20	RGB LED PWM	R, G, B 버튼으로 세 PWM 듀티비를 독립적으로 변경
21	피에조 단일 음	50MHz 클록을 분주하여 약 294Hz 사각파 생성
22	스텝모터 위상 제어	4상 코일 패턴을 순서대로 출력하여 정, 역회전 제어
23	MM: SS 시계	00:00부터 59:59까지 계수하고 7세그먼트로 표시
24	문자 LCD 제어	초기화 명령 전송 후 두 줄 문자열 표시
25	PC-FPGA UART 에코	9600 8N1로 수신한 데이터를 재전송하고 LED에 표시

2. 관련 이론

단일 클록과 clock-enable

LAB3의 모든 회로는 50MHz 시스템 클록을 기준으로 동작한다. 50MHz 클록의 주기는 20ns이며, PWM 주기, 피

에조출력, 스텝모터 상태 전환, LCD 제어 시간은 카운터를 이용해 생성한다. 느린 동작이 필요하더라도 카운터 출력을 새로운 클록으로 사용하지 않는다. 모든 순차회로는 정해진 클록의 상승 에지에서 동작하고, 카운터가 설정 값에 도달했을 때만 상태나 출력을 변경한다. 이러한 동기식 방식은 회로 전체가 하나의 클록 영역에서 동작하도록 하므로 클록 간 타이밍 문제를 줄이고 FPGA의 타이밍 분석을 단순하게 한다.

원하는 이벤트 주파수가 Fout일 때 필요한 카운트 수는 일반적으로 다음과 같다.

$$N = \text{CLK_Hz} / \text{Fout}$$

출력을 반주기마다 반전하여 사각파를 생성할 때는 다음 식을 사용한다.

$$\text{HALF_PERIOD} = \text{CLK_Hz} / (2 \times \text{Fout})$$

PWM 듀티비와 LED 밝기

PWM은 일정한 주기 안에서 출력이 HIGH인 시간의 비율을 변경하는 방법이다. 듀티비는 전체 주기 대비 HIGH인 시간의 백분율 표현으로 나타낸다. 실험 19와 20에서는 PWM 카운터가 임계 값보다 작은 동안 출력을 HIGH로 만든다. 임계 값(threshold)는 다음과 같이 level에 비례하여 계산된다.

$$\text{Threshold} = \text{PERIOD_CYCLES} \times \text{level} / \text{LEVELS}$$

예를 들어 LEVELS=10이면 level이 1 증가할 때 마다 듀티비가 10%씩 증가한다. LED는 PWM의 반복 속도가 충분히 빠르면 각각의 ON/OFF를 구분하기 어렵고 평균 밝기로 인식된다. 듀티비가 증가하면 한 주기 동안 LED에 전류가 흐르는 시간이 길어지므로 평균 전력과 밝기가 증가한다. 다만 사람의 밝기 인식과 LED의 광출력이 선형적이지 않으므로, 듀티비가 2배가 되었다고 해서 사람이 느끼는 밝기 또한 2배가 되는 것은 아니다.[\[1\]](#)

3. 개발 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
코드 작성	VS Code 1.137.0
HDL Simulator	Icarus Verilog 13.0
파형 확인	VaporView
FPGA Part	Xc7s75fpga484-1
실행 Script	Python 3.14.7
FPGA 도구	AMD Vivado 2026.1
보드	Combo II-DLD S75
시스템 클록	MAIN CLOCK F, 50MHz

파일의 역할

RTL 파일은 FPGA에 합성될 실제 회로를 기술한다. 기능별 하위 모듈은 버튼 입력 처리, PWM 생성, 시간 계수, 표시장치 구동 및 UART 송수신을 담당하며, 각 실험의 최상위 RTL 모듈을 Vivado의 Design Top으로 설정한다.

테스트 벤치는 DUT에 클록, 리셋, 버튼 및 직렬 입력을 가하고 실제 출력과 고정된 기대값을 비교한다. 결과가 다르면 \$fatal로 종료하고, 검사가 통과되면 실험별로 PASS 문자열을 출력한다. 또한, wave.vcd를 생성한다.

XDC 파일은 Design Top의 포트를 FPGA 패키지 핀에 연결하고 모든 포트에 LVCMOS33을 지정한다. XDC는 시뮬레이션에서는 사용되지 않으며 실제 핀, 입출력 규격 및 timing 분석을 위해 Vivado에서 사용한다.

Simulation.json은 Icarus가 읽을 RTL 파일, Tb 파일, Simulation top을 지정한다.

Vivado 등록 항목

실험	Design Top	Simul Top	XDC
19	lab3_led_pwm	tb_led_pwm	lab3_led_pwm.xdc
20	lab3_rgb_pwm	tb_rgb_pwm	lab3_rgb_pwm.xdc
21	lab3_piezo	tb_piezo	lab3_piezo.xdc
22	lab3_stepper	tb_stepper	lab3_stepper.xdc
23	lab3_mmss_clock	tb_mmss_counter	lab3_mmss_clock.xdc
24	lab3_character_lcd	tb_character_lcd	lab3_character_lcd.xdc
25	lab3_uart_echo	tb_uart_echo	lab3_uart_echo.xdc

4. 실험 19 · PWM LED 밝기

핵심 내용

·입력: clk_50Mhz, rst_p, button

·출력: led[7:0]

·정상 로그: LAB3_LED_PWM_PASS checks=4

·블록 흐름:

button – 2단 동기화, 디바운스 – press 1클록 펄스 – level 증가 – PWM 비교기 – 동일한 PWM을 LED 8개에 복제

파라미터 계산

·CLK_HZ = 50,000,000 ·PWM_HZ = 1,000

·PERIOD_CYCLES = 50,000,000 / 1,000 = 50,000

·PWM 주기: 50,000 × 20 ns = 1 ms

·LEVELS = 10: 한 단계당 10%

·DEBOUNCE_CYCLES = 1,000,000: 20 ms

Threshold = PERIOD_CYCLES × level / LEVELS

핵심 코드

```
always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p)
        level <= 4'd0;
    else if (press)
        level <= (level == LEVELS) ? 4'd0 : level + 1'b1;
end
```

```
threshold = (PERIOD_CYCLES * level) / LEVELS;
```

```
pwm <= (count < threshold);
```

그림 2 : 실험 19 Verilog 핵심 코드

버튼의 유효 press가 발생할 때 마다 level 1씩 증가한다. level이 LEVELS인 상태에서 다시 버튼을 누르면 0으로 돌아간다. PWM 카운터 임계 값보다 작은 동안 출력이 1이 되며, 동일한 PWM 신호를 8개 LED에 출력한다.

예상 결과

리셋 직후 level은 0이므로 LED가 꺼진다. 버튼을 누를 때 마다 듀티비가 10%씩 증가한다. level 3에서는 30%, level 10에서는 100%의 듀티비가 출력된다. level 10에서 버튼을 한 번 더 누르면 level이 0으로 돌아가 LED가 꺼진다.

TB 자극과 기대 결과

순서	자극	기대결과
1	리셋 해체	HIGH 수 0
2	버튼 3회	HIGH 수 3
3	버튼 7회 추가	HIGH 수 10
4	버튼 1회 추가	HIGH 수 0으로 순환

결과 설명

```
Run C:\msys64\mingw64\bin\vvp.EXE -N D:\Engineering\Verilog_Lab\lab3_19_led_pwm\build\sim-run-b543dcbe805f4bd18a3183b0a47f0e59\sim_vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_LED_PWM_PASS checks=4
D:\Engineering\Verilog_Lab\lab3_19_led_pwm\sim\tb_led_pwm.sv:88: $finish called at 3831000 (1ps)
```

그림 3 : 실험 19 시뮬레이션 실행 결과(PASS)

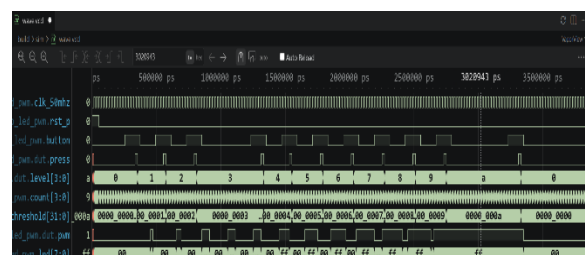


그림 4 : 실험 19 PWM 정상 파형

Icarus 시뮬레이션에서 LAB3_LED_PWM_PASS checks=4가 출력되었으며 종료 시각은 3,831,000 ps였다. 파형에서 level이 0, 3, 10, 0 순서로 변경되었고, PWM 한 주기 10클록 중 HIGH 구간이 각각 0, 3, 10, 0클록으로 나타났다.

5. 실험 20 · RGB LED PWM

핵심 내용

·입력: clk_50mhz, rst_p, button_r, button_g, button_b

·출력: led_r[3:0], led_g[3:0], led_b[3:0]

·정상 로그: LAB3_RGB_PWM_PASS checks=2

·블록 흐름:

R/G/B 버튼 – 채널별 원펄스 – Level_r/g/b – 공통 주기의 PWM 세 채널 – 네 개 RGB LED의 R/G/B 핀

파라미터 계산

(실제회로: PERIOD_CYCLES = 50000)

·PWM 주기: 50,000클록 = 1 ms

(가속 TB: PERIOD_CYCLE = 10)

·R 20%: 10클록 중 2클록 HIGH

·G 50%: 10클록 중 5클록 HIGH

·B 80%: 10클록 중 8클록 HIGH

핵심 코드

```
always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        level_r <= 0;
        level_g <= 0;
        level_b <= 0;
    end else begin
        if (press_r)
            level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;

        if (press_g)
            level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;

        if (press_b)
            level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
    end
end
```

그림 5: 실험 20 Verilog 핵심 코드

각 press는 해당 색의 level만 증가시킨다. 세 PWM 채널은 서로 다른 threshold를 사용하므로 HIGH 폭이 독립적으로 결정된다.

예상 결과

R 버튼을 2회, G 버튼을 5회, B 버튼을 8회 누르면 각 색의 PWM 듀티비는 20%, 50%, 80%가 된다. 이후 R 버튼을 한 번 더 누르면 R만 30%로 증가하고 G와 B는 각각 50%,

80%로 유지된다.

TB 자극과 기대 결과

TB 자극	R HIGH	G HIGH	B HIGH
리셋 직후	0	0	0
R 2회, G 5회, B 8회	2클록	5클록	8클록
R 1회 추가	3클록	5클록	8클록

결과 설명

```
RUN C:\msys64\mingw64\bin\iycp.EXE -N D:\Engineering\Verilog_Lab\Lab3_20_rgb_pwm\build\sim-run-678caab70664140644021b3a69fcae\sim.vvp
VCD info: dumpFile wave.vcd opened for output.
LAB3_RGB_PWM_PASS checks=2
D:\Engineering\Verilog_Lab\Lab3_20_rgb_pwm\sim\tb_rgb_pwm.vv:189: $finish called at 4431000 (1ps)
```

그림 6: 실험 20 시뮬레이션 실행 결과(PASS)



그림 7: 실험 20 RGB PWM 정상 파형

Icarus 시뮬레이션에서 LAB3_RGB_PWM_PASS checks=2가 출력되었으며 종료 시각은 4,431,000 ps였다. 정상 파형에서 처음에는 R, G, B가 각각 20%, 50%, 80%의 HIGH 폭을 가졌고, R 버튼을 한 번 더 누른 뒤에는 R만 30%로 증가하고 G와 B는 유지되었다.

6. 실험 21 · 피에조 단일 음

핵심 내용

·입력: clk_50mhz, rst_p ·출력: piezo

·정상로그: LAB3_PIEZO_PASS edges=5

·블록 흐름:

50MHz 클록 – 반주기 카운터 – HALF_PERIOD마다 출력 반전 – 294Hz 사각파 – 피에조

파라미터 계산

·CLK_HZ = 50,000,000 ·TONE_HZ = 294

·HALF_PERIOD = 50,000,000 / (2 × 294) = 85,034클록

·반주기 시간: 85,034 × 20 ns = 1.70068 ms

핵심 코드

```

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        count <= 0;
        piezo <= 0;
    end else if (count == HALF_PERIOD - 1) begin
        count <= 0;
        piezo <= ~piezo;
    end else begin
        count <= count + 1'b1;
    end
end

```

그림 8: 실험 21 Verilog 핵심 코드

예상 결과

실제 보드에서는 piezo가 약 1.70068 ms마다 반전되어 약 294 Hz의 단일 음을 발생시킨다. TB에서는 count가 0부터 4까지 진행한 뒤 piezo가 반전되어야 한다.

TB 자극과 기대 결과

TB 자극 및 검사	기대 결과
리셋 3클록	count=0, piezo=0
리셋 해제 후 5클록	Piezo 첫 반전
이후 5클록	Piezo 다시 반전
25클록 실행	반전 간격 5클록, 총 edges 5

결과 설명

```

RUN C:\msys64\mingw64\bin\vpj.exe -U D:\Engineering\Verilog_Lab\area2\lab3\lab3_21_piezo\build\sim\run-6371b65783b4%41be331f6db398000\sim.vvp
VCD info: dumpFile wave.vcd opened for output.
LAB3_PIEZO_PASS edges=5
D:\Engineering\Verilog_Lab\area2\lab3\lab3_21_piezo\sim\th_piezo.sv:41: $finish called at 551000 (ps)

```

그림 9: 실험 21 시뮬레이션 실행 결과 (PASS)

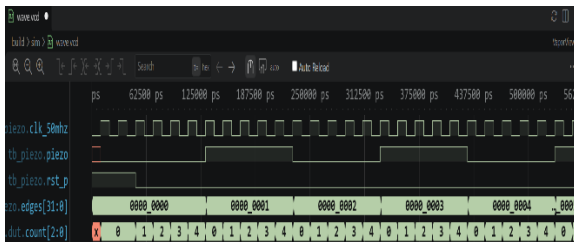


그림 10: 실험 21 피에조 정상 파형

Icarus 시뮬레이션에서 LAB3_PIEZO_PASS edges=5가 출력되었으며 종료 시각은 551,000ps였다. 파형에서 count가 0부터 4까지 진행한 후 0으로 돌아갈 때마다 piezo가 반전되었다.

7. 실험 22 · 스텝모터 위상 제어

핵심 내용

·입력: clk_50mhz, rst_p, enable, direction

·출력: stepmotor[3:0]

·정상 로그: LAB3_STEPPER_PASS checks=8

·블록 흐름:

Enable/direction 2단 동기화 - 스텝 카운터 - state 증가 또는 감소 - 4상 여자 패턴 출력

파라미터 계산

·CLK_HZ = 50,000,000

·STEP_HZ = 100

·STEP_CYCLES = 50,000,000 / 100 = 500,000

·한 단계 유지 시간: 500,000 × 20 ns = 10 ms

핵심 코드

```

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        count <= 0;
        state <= 0;
    end else if (!enable_sync) begin
        count <= 0;
    end else if (count == STEP_CYCLES - 1) begin
        count <= 0;
        state <= direction_sync ? state - 1'b1
                                : state + 1'b1;
    end else begin
        count <= count + 1'b1;
    end
end

```

```

always @* begin
    case (state)
        2'd0: stepmotor = 4'b0011;
        2'd1: stepmotor = 4'b0110;
        2'd2: stepmotor = 4'b1100;
        default: stepmotor = 4'b1001;
    endcase
end

```

그림 11: 실험 22 Verilog 핵심 코드

state 0부터 3까지 각각 네 가지 여자 패턴을 출력한다. direction은 state의 증가 또는 감소 방향을 결정한다.

예상 결과

정방향에서는 0011 → 0110 → 1100 → 1001 → 0011 순서로 출력된다. 역방향에서는 반대 순서로 진행한다. enable을 0으로 만들면 마지막 state와 출력 패턴이 유지된다.

TB 자극과 기대 결과

단계	Enable	Direction	기대 출력
리셋 직후	0	0	0011

정방향 1단계	1	0	0110
정방향 2단계	1	0	1100
정방향 3단계	1	0	1001
정방향 4단계	1	0	0011
역방향 1단계	1	1	1001
역방향 2단계	1	1	1100
정지	0	X	1100 유지

결과 설명

```

RUN C:\msys64\mingw64\bin\vvp.EXE -N D:\Engineering\Verilog_Lab\ec2-Lab3\Lab3_22_stepper\build\sin\run-4633856ac4e34986582239604492033c\sin.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_STEPPER_PASS checks=8
D:\Engineering\Verilog_Lab\ec2-Lab3\Lab3_22_stepper\sin\th_stepper.sv:72: $finish called at 831000 (ps)

```

그림 12: 실험 22 시뮬레이션 실행 결과(PASS)

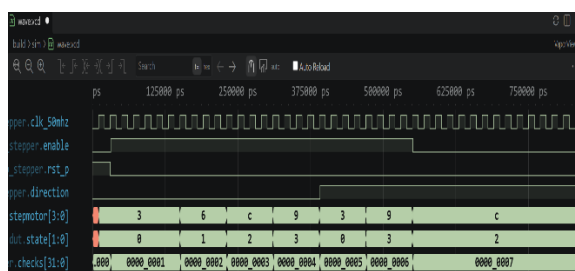


그림 13: 실험 22 스텝 모터 정상 파형

LAB3_STEPPER_PASS checks=8이 출력되었으며 종료 시간은 831,000 ps였다. enable이 1이 된 뒤 state가 4클럭마다 증가하며 출력 패턴이 정방향으로 진행하였다. direction이 1로 동기화된 뒤에는 state가 감소하여 역방향 패턴이 나타났다. enable을 0으로 변경한 뒤에는 1100이 유지되었다.

8. 실험 23 · MM:SS 시계

핵심 내용

·입력: clk_50mhz, rst_p

·출력: seg_data[7:0], seg_com[7:0]

·정상 로그: LAB3_MMSS_PASS checks=7

·블록 흐름:

1초 카운터 – MM BCD 카운터 – 7세그먼트 디코더 – 4자리 동적 스캔

파라미터 계산

·1초: 50,000,000클럭

·SCAN_HZ = 4,000

·SCAN_CYCLES = 50,000,000 / 4,000 = 12,500

·한 자리 선택 시간: 12,500 × 20 ns = 250 μs

·네 자리 전체 스캔 시간: 250 μs × 4 = 1 ms

핵심 코드

```

end else if (subsecond == CLK_HZ - 1) begin
    subsecond <= 0;

    if (second_ones != 9)
        second_ones <= second_ones + 1'b1;
    else begin
        second_ones <= 0;

        if (second_tens != 5)
            second_tens <= second_tens + 1'b1;
        else begin
            second_tens <= 0;

            if (minute_ones != 9)
                minute_ones <= minute_ones + 1'b1;
            else begin
                minute_ones <= 0;

                if (minute_tens != 5)
                    minute_tens <= minute_tens + 1'b1;
                else
                    minute_tens <= 0;
            end
        end
    end
end else begin
    subsecond <= subsecond + 1'b1;
end
end

```

그림 14: 실험 23 Verilog 핵심 코드

초의 일의 자리가 9에 도달하면 0으로 돌아가고 초의 십의 자리가 증가한다. 초의 십의 자리가 5에 도달하면 초는 00으로 초기화되고 분이 증가한다. 분도 같은 방식으로 자리올림 된다.

예상 결과

리셋 직후 시간은 00:00이다. 10초 후에는 00:10, 총 60초 후에는 01:00이 된다. 59:59에서 1초가 지나면 00:00으로 순환한다.

TB 자극과 기대 결과

검사	기대 결과
숫자 0 디코딩	Seg_data=FC
숫자 9 디코딩	Seg_data=F6
리셋 직후	00:00
10초 진행	00:10
50초 추가	01:00
3,539초 추가	59:59
1초 추가	00:00

결과 설명

```

RUN C:\msys64\mingw64\bin\vvp.EXE -N D:\Engineering\Verilog_Lab\ec2-Lab3\Lab3_23_mss_clock\build\sin\run-5c8d3ee8c234e59373e60c1d0f1f0\sin.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_MMSS_PASS checks=7
D:\Engineering\Verilog_Lab\ec2-Lab3\Lab3_23_mss_clock\sin\th_mss_counter.sv:86: $finish called at 144631000 (ps)

```

그림 15: 실험 23 시뮬레이션 실행 결과(PASS)

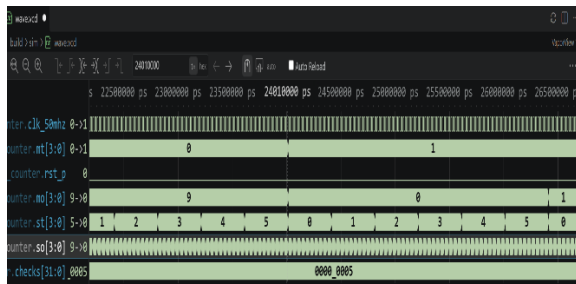


그림 16: 실험 23 MM: SS 시계 정상 파형

LAB3_MMSS_PASS checks=7이 출력되었으며 종료 시각은 144,031,000ps였다. 디코더 패턴과 00:00 → 00:10 → 01:00 → 59:59 → 00:00의 경계 조건이 모두 통과하였다.

9. 실험 24 · 문자 LCD 제어

핵심 내용

·입력: clk_50mhz, rst_p

·출력: lcd_e, lcd_rs, lcd_rw, lcd_data[7:0]

·정상 로그: LAB3_LCD_PASS bytes=40

பிழைப்பு

10 μ s tick - 전원 안정화 대기 - 명령 및 문자 선택 - E 펄스 - 명령별 대기

파라미터 계산

```
·TICK_CYCLES = 500
```

```
·POWER_TICKS = 2,000: 20 ms
```

·NORMAL WAIT TICKS = 4: 40 μ s

- CLEAR_WAIT_TICKS = 200: 2 ms

핵심 코드

```

case (phase)
0: begin
    lcd_e <= 0;
    lcd_rs <= byte_rs;
    lcd_data <= byte_data;
    phase <= 1;
end

1: begin
    lcd_e <= 1;
    phase <= 2;
end

2: begin
    lcd_e <= 0;
    wait_count <=
        (index == 5) ?
            CLEAR_WAIT_TICKS :
            NORMAL_WAIT_TICKS;
    phase <= 3;
end

default: begin
    if (wait_count > 0)
        wait_count <= wait_count - 1;
    else begin
        index <=
            (index == 39) ?
                6 :
                index + 1'b1;
        phase <= 0;
    end
end
end

```

그림 17: 실험 24 Verilog 핵심 코드

phase 0에서 RS와 데이터를 준비하고, phase 1에서 E를 1로 만든다. phase 2에서 E를 0으로 내리면서 해당 바이트를 LCD에 전달하고 명령 종류에 맞는 대기 시간을 설정한다.

예상 결과

전원 안정화 대기 후 LCD 초기화 명령 38, 38, 38, 0C, 06, 01이 순서대로 전송된다. 이어서 80 명령으로 첫째 줄의 시작 주소를 선택하고 FPGA LAB3와 남은 공백을 전송한다. 이후 C0 명령으로 둘째 줄을 선택하고 LCD CONTROLLER와 공백을 전송한다.

명령을 전송할 때는 lcd_rs=0, 문자 데이터를 전송할 때는 lcd_rs=1이 되며, 쓰기 동작만 수행하므로 lcd_rw=0을 유지할 것으로 예상된다. 각 바이트는 lcd_e의 하강 에지에서 LCD로 전달되며, 총 40바이트가 순서대로 출력되어야 한다.

TB 자극과 기대 결과

검사구간	기대 결과
초기화	38-38-38-0C-06-01
첫째 줄	80 전송 후 FPGA LAB3 출력
둘째 줄	C0 전송 후 LCD CONTROLLER 출력
제어 신호	명령은 RS=0, 문자는 RS=1, RW=0 유지
완료 조건	총 40바이트 일치 후 LAB3 LCD PASS bytes=40

결과 설명

```

D:\C:\msys64\mingw64\bin\wpd.exe -D D:\Engineering\Verilog_Lab\lab3\lab3_24_character_lcd\build\sin\wpd-99ca363636a4ae6f3c321f57c74680c2\sin.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_LCD_PASS bytes=0
D:\Engineering\Verilog_Lab\lab3\lab3_24_character_lcd\src\character_lcd.v:101: $finish called at 1030000 (top)

```

그림 18: 실험 24 시뮬레이션 실행 결과(PASS)

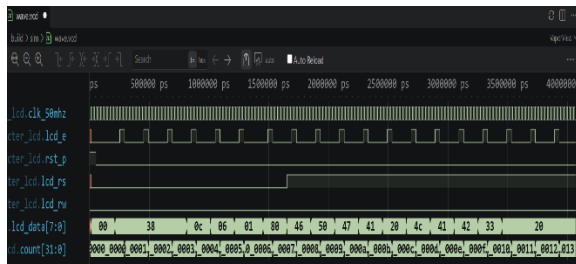


그림 19: 실험 24 문자 LCD 정상 파형

LAB3_LCD_PASS bytes=40이 출력되었으며 종료 시각은 8,130,000 ps였다. 파형에서 초기화 명령 뒤 FPGA LAB3, C0, LCD CONTROLLER 순서로 데이터가 출력되었다. 명령에서는 RS가 0, 문자에서는 RS가 1이었으며 RW는 항상 0이었다.

10. 실험 25 · PC FPGA UART 예코

핵심 내용

·입력: clk_50mhz, rst_p, uart_rxd

·출력: uart_txd, led[7:0]

·통신 조건: 9600 bps, 8N1

·정상 로그: LAB3_UART_ECHO_PASS checks=3

·블록 흐름:

RX 2단 동기화 – start 중앙 확인 - 8비트 LSB-first 수신 – stop 검사 – LED 저장 및 TX echo

파라미터 계산

·CLK_HZ = 50,000,000

·BAUD = 9,600

·DIV = (50,000,000 + 4,800) / 9,600 = 5,208

·한 비트 시간: $5,208 \times 20 \text{ ns} = 104.16 \mu\text{s}$

핵심 코드

```
always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        tx_valid <= 0;
        tx_data <= 0;
        last_data <= 0;
    end else begin
        tx_valid <= 0;

        if (rx_valid) begin
            last_data <= rx_data;

            if (tx_ready) begin
                tx_data <= rx_data;
                tx_valid <= 1;
            end
        end
    end
end
```

그림 20: 실험 25 Verilog 핵심 코드

RX 수신 완료 시 데이터를 LED 저장 레지스터에 기록한다. TX가 준비되어 있으면 동일한 데이터를 TX로 전달하여 echo를 시작한다.

예상 결과

TB가 41, 5A, 0A를 차례로 전송하면 TX에서도 같은 값이 반환되어야 한다. LED는 마지막으로 수신한 데이터로 갱신되어야 하며 정상 stop bit를 사용하므로 framing error가 발생하지 않아야 한다

TB 자극과 기대 결과

RX 전송	기대 TX Echo	기대 LED	Framing Error
41	41	41	0
5A	5A	5A	0
0A	0A	0A	0

결과 설명

```
Run C:\msys64\mingw64\bin\vvp.EXE -I D:\Engineering\Verilog_Lab\ex2-Lab3\Lab3_uart_echo\build\sin\run-73faad8a5e48fcb63141621097e4d\sin.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_UART_ECHO_PASS checks=3
D:\Engineering\Verilog_Lab\ex2-Lab3\Lab3_uart_echo\sin\tb_uart_echo.sv:188: $finish called at 9970000 (1ps)
```

그림 21: 실험 25 시뮬레이션 실행 결과(PASS)

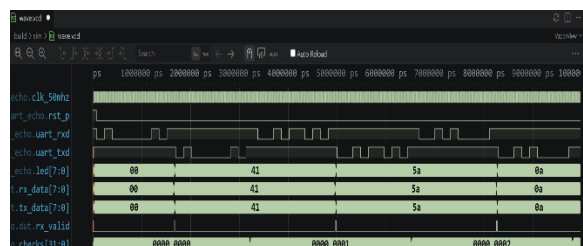


그림 22: 실험 25 UART Echo 정상 파형

LAB3_UART_ECHO_PASS checks=3이 출력되었으며 종료

시각은 9,970,000 ps였다. 41, 5A, 0A가 각각 동일한 값으로 echo되었고 LED에도 수신값이 표시되었다. 모든 프레임에서 정상 stop bit가 확인되어 framing error가 발생하지 않았다.

11. 정상 수정 복구 검증

기본적으로 정상 - 수정 - 복구 검증의 과정은 다음 표와 같은 단계를 거쳤고, 결과 및 증빙은 다음과 같다. 예외적으로 실험 22와 25처럼 주파수만 변경하고 기능 순서가 유지되는 실험은 FAIL이 아니라 변경된 타이밍과 PASS를 확인하였다.

단계	RTL 상태	결과 및 증빙
정상	원래 코드	PASS 로그, 정상 파형
수정	지정 부분 의도적 변경	수정 로그 및 수정 파형
복구	원상 복구	최종 PASS 및 정상 소스

실험 19 수정 결과

단계	설정	결과
정상	LEVELS=10	Level 3에서 HIGH 3클록, PASS
수정	LEVELS=5	Level 3에서 HIGH 6클록, FAIL
복구	LEVELS=10	기존 듀티비와 PASS 복구

```

RUN C:\msys64\mingw64\bin\wvp.EXE -N D:\Engineering\Verilog_Lab3\lab3_19_led_pwm\build\sim-run-5a2d6e8449214567650401783149904\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
FATAL: D:\Engineering\Verilog_Lab3\lab3_19_led_pwm\sim\led_pwm.sv:53: level=3 high=4
Time: 1231800 Scope: tb_led_pwm.expect_level
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab3\lab3_19_led_pwm\build\sim-run-5a2d6e8449214567650401783149904\simulation.log
  
```

그림 23: 실험 19 시뮬레이션 실행 결과(FAIL)

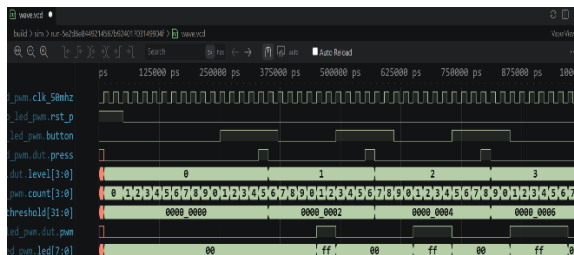


그림 24: 실험 19 수정 파형

실험 20 수정 결과

단계	R 입력	실제 HIGH 수	결과
정상	R 버튼 2회	2, 5, 8	PASS
수정	R 버튼 4회	4, 5, 8	FAIL
복구	R 버튼 2회	2, 5, 8	PASS 복구

```

RUN C:\msys64\mingw64\bin\wvp.EXE -N D:\Engineering\Verilog_Lab3\lab3_20_rgb_pwm\build\sim-run-39c83d118214f39f65e3773becf8\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
FATAL: D:\Engineering\Verilog_Lab3\lab3_20_rgb_pwm\sim\rgb_pwm.sv:87: rgb_high=4,5,8
Time: 4431000 Scope: tb_rgb_pwm.measure
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab3\lab3_20_rgb_pwm\build\sim-run-39c83d118214f39f65e3773becf8\simulation.log
  
```

그림 25: 실험 20 시뮬레이션 실행 결과(FAIL)

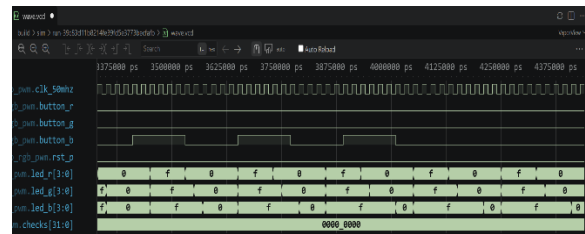


그림 26: 실험 20 수정 파형

실험 21 수정 결과

단계	TONE_HZ	반주기	결과
정상	100	5클록	PASS
수정	125	4클록	FAIL
복구	100	5클록	PASS 복구

```

RUN C:\msys64\mingw64\bin\wvp.EXE -N D:\Engineering\Verilog_Lab3\lab3_21_piano\build\sim-run-66736e3ac8445369c3a959e4e78f\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
FATAL: D:\Engineering\Verilog_Lab3\lab3_21_piano\sim\piano.sv:32: half_period=4
Time: 195000 Scope: tb_piano
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab3\lab3_21_piano\build\sim-run-66736e3ac8445369c3a959e4e78f\simulation.log
  
```

그림 27: 실험 21 시뮬레이션 실행 결과(FAIL)



그림 28: 실험 21 수정 파형

실험 22 수정 결과

단계	STEP_HZ	상태 전환 간격	결과
정상	2	4클록	PASS
수정	1	8클록	PASS, 동작 속도 감소
복구	2	4클록	PASS 복구

```

RUN C:\msys64\mingw64\bin\wvp.EXE -N D:\Engineering\Verilog_Lab3\lab3_22_stepper\build\sim-run-f9d5c31a2f43f5a8b39a7c980f9a5\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_STEPPER_PASS checks=8
D:\Engineering\Verilog_Lab3\lab3_22_stepper\sim\stepper.sv:72: Finish called at 1311000 (1ps)
  
```

그림 29: 실험 22 시뮬레이션 실행 결과(수정)

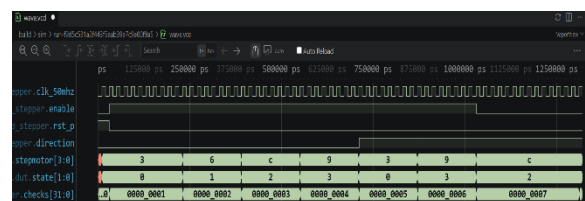


그림 30: 실험 22 수정 파형

실험 23 수정 결과

단계	DUT CLK_HZ	TB가 10초로 판단한 시점	결과
정상	2	00:10	PASS
수정	4	00:05	FAIL
복구	2	00:10	PASS 복구

```

RUN C:\msys64\mingw64\bin\vp.exe -N D:\Engineering\Verilog_Lab\ecol-lab3\lab3_23_mms_clock\build\sim-run-d88d6f9c2b74491893adF2691af06a\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
FATAL: D:\Engineering\Verilog_Lab\ecol-lab3\lab3_23_mms_clock\sim\th_mms_counter.sv:47: time=00:05
Time: 4310000 Scope: th_mms_counter.check_time
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ecol-lab3\lab3_23_mms_clock\build\sim-run-d88d6f9c2b74491893adF2691af06a\simulation.log

```

그림 31: 실험 23 시뮬레이션 실행 결과(FAIL)

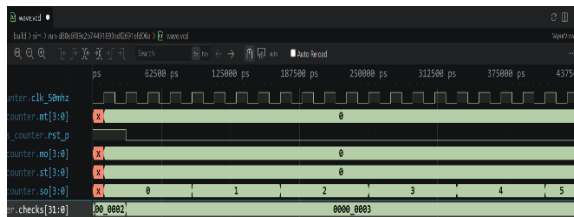


그림 32: 실험 23 수정 파형

실험 24 수정 결과

단계	Index 24	ASCII	결과
정상	L	4C	PASS
수정	X	58	FAIL
복구	L	4C	PASS 복구

```

RUN C:\msys64\mingw64\bin\vp.exe -N D:\Engineering\Verilog_Lab\ecol-lab3\lab3_24_character_lcd\build\sim-run-527a1b7c794946477ad0e0b32c\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
FATAL: D:\Engineering\Verilog_Lab\ecol-lab3\lab3_24_character_lcd\sim\th_character_lcd.sv:88: Index24 rest data=58
Time: 5130000 Scope: th_character_lcd
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ecol-lab3\lab3_24_character_lcd\build\sim-run-527a1b7c794946477ad0e0b32c\simulation.log

```

그림 33: 실험 24 시뮬레이션 실행 결과(FAIL)

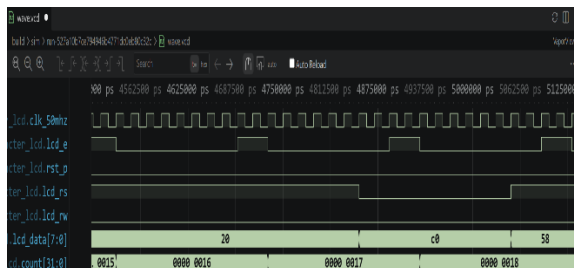


그림 34: 실험 24 수정 파형

실험 25 수정 결과

단계	BAUD	DIV	비트 시간	결과
정상	100	8	160ns	PASS
수정	200	4	80ns	PASS
복구	100	8	160ns	PASS 복구

```

RUN C:\msys64\mingw64\bin\vp.exe -N D:\Engineering\Verilog_Lab\ecol-lab3\lab3_25_uart_echo\build\sim-run-c59a3a4618a618a2af9b35476774\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB3_UART_echo_PASS check=3
D:\Engineering\Verilog_Lab\ecol-lab3\lab3_25_uart_echo\sim\th_uart_echo.sv:108: Finish called at 5170000 (ps)

```

그림 35: 실험 25 시뮬레이션 실행 결과(PASS)

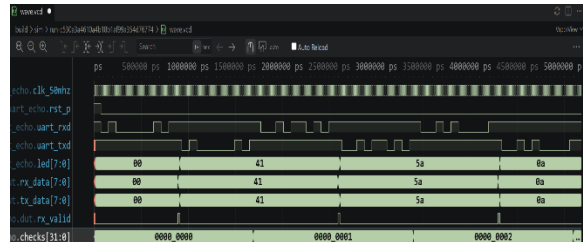


그림 36: 실험 25 수정 파형

최종 정상 코드 커밋 및 PASS 증빙

모든 수정 항목을 정상 값으로 복구한 뒤 정상 시뮬레이션을 다시 실행하였다. 최종 저장소에는 복구 정상 RTL과 TB만을 남겼다. 실험19-25를 한 번에 커밋하였기에 커밋 주소는 아래와 같고, 모두 동일하다.

(커밋: [26acfe878b6d7e0b0355c3be7cb1d638fb124d1c](#))

12. 보드 검증 계획

실제 실험에서는 동일한 RTL과 XDC를 Vivado에 등록한다. Design Top과 Simulation Top을 구분하고, XSim 기능 시뮬레이션에서 Icarus와 같은 PASS 문자열 및 파형을 확인한다. 이후 합성, 구현, DRC, 타이밍 및 비트스트림 생성을 수행한다. 이후 FPGA 보드에서 입력 조작과 출력 변화가 함께 보이는 사진 및 영상을 기록한다.

실험	보드에서 확인할 항목
19	버튼을 반복해서 누르면 LED 밝기 관찰
20	R, G, B 버튼을 각각 조작
21	피에조 출력 확인
22	Enable과 direction 변경
23	7세그먼트 시간 증가 관찰
24	LCD 전원·논리전압·대비 확인 후 문자열 관찰
25	PC 터미널을 9600 8N1, local echo off로 설정

13. 결론

LAB3에서는 앞선 수업에서 다룬 기본적인 조합 및 순차논리 회로를 기준으로 일곱 종류의 FPGA 응용회로를 설계하였다. PWM LED와 RGB LED에서는 카운터와 임계값 비교를 이용하여 듀티비를 조절하였다. 피에조에서는 반주기마다 출력을 반전하여 사각파를 만들었고, 스텝모터에서는 네 가지 여자 패턴과 direction 입력을 이용하여 회전 방향을 결정하였다. 그리고 MM: SS 시계에서는 1초 카운터와 BCD 자리올림을 구현하고 7세그먼트 스캔으로 네 자리를

표시하였다. 문자 LCD에서는 초기화 명령, 문자 데이터, enable 펄스 및 명령별 대기 시간을 순서대로 제어하였고, UART에서는 비동기 입력을 동기화하고 데이터 수신 및 stop bit 확인을 수행한 뒤 같은 데이터를 다시 송신하였다. 이처럼 앞서 배운 회로를 응용하여 다양한 회로를 설계한 것이다. 시뮬레이션을 통해 파형을 확인하고, 앞선 수업들과 동일하게 Vivado를 사용하여 실제 보드에서 시연한 후 응용회로의 동작을 확인하고, 분석해볼 예정이다.

참고 문헌

[1] Texas Instruments, "How to Select a RGB LED Driver," Application Report SLVAEF3, Aug. 2019, pp. 5–6.